



Zoho Schools for Graduate Studies



Notes

Constructor Invocation in Java

Super() :

The **super()** keyword in Java is used to **call the constructor of the parent (superclass)** from a **child (subclass)**.

It ensures that the parent class is properly initialized before the child class adds its own initializations.

//Multilevel inheritance

ex.1

In Java, an implicit **super()** **is automatically inserted by the compiler** as the **first line of a child class constructor** if and only if the constructor doesn't have any form of **super()** or **this()** call explicitly by programmer.

note:

- implicitly provided **super()** call statement will always invoke the zero argument constructor of the super class.
- **super** keyword can be used by a sub class to access its super class members

// Parent class

```
class Game{
```

```
String type;
```

```
    Game(){
```

```
        System.out.println("Game"); // This prints "Game"
```

```
    }
```

```
}
```

// Child class of Game

```
class IndoorGame extends Game{
```

```
    IndoorGame(){
```

```
        // Implicit super() → calls Game() constructor
```

```
        System.out.println("IndoorGame"); // Prints "IndoorGame"
```

```
    }
```

```
}
```

// Child class of IndoorGame

```
class Chess extends IndoorGame{
```

```

    Chess(){
        super(); //explicitly calls IndoorGame() constructor
        System.out.println("Chess"); // Prints "Chess"
    }
}

public class InheritanceDemo{
    public static void main(String[] args){
        Chess c = new Chess(); // output :
                                Game
                                IndoorGame
                                Chess
    }
}

```

ex.2

Object class : (ultimate super class)

- Object is the **root class** of **all classes** in Java which is present in **java.lang** package.
- Every class in Java **directly or indirectly inherits** from Object.
- Even if you don't write extends Object explicitly. it is implicitly inherits by subclass.

note :

- in java , Orphan classes are not allowed except Object class
- if a class doesn't inherit any other class , then the class will be made to inherit Object class implicitly.
- in java , every class must inherit one another class except Object class.

// child class of Object class

```
class Game extends Object{ // either implicit or explicitly inherited (Object class)
```

String type;

```

    Game(){
        // Implicit super() → calls Object() constructor
        System.out.println("Game");
    }
}

```

ex.3

this() : this() call is used to invoke another constructor within same class.

note :

- When you use this() in a constructor, it **must be explicitly written** by the programmer. Unlike super(), which is **implicitly added** by the compiler if you don't write it, this() is not provided implicitly.
- **Must be the first statement** in the constructor.
- this() and super() call **cannot co-exist** in a constructor.
- **this** is a reference to the **current instance** of the class. used to distinguish between instance variables and parameters.

// Parent class

```
class Game{
```

```
String type;
```

```
    Game(){
```

```
        System.out.println("Game"); // This prints "Game"
```

```
    }
```

```
}
```

// Child class of Game

```
class IndoorGame extends Game{
```

```
    IndoorGame(){
```

```
        // Implicit super() → calls Game() constructor
```

```
        System.out.println("IndoorGame"); // Prints "IndoorGame"
```

```
    }
```

```
}
```

// Child class of IndoorGame

```
class Chess extends IndoorGame{
```

```
    Chess(){
```

```
        this(12); //call Chess() constructor which accepts int as an arguments within same class
```

```
        System.out.println("Chess"); // Prints "Chess"
```

```
    }
```

```
    Chess(int x){
```

```
        // Implicit super() → calls IndoorGame() constructor
```

```
    }
```

```

}

public class InheritanceDemo{

    public static void main(String[] args){

        Chess c = new Chess(); // output :

                                   Game
                                   IndoorGame
                                   Chess

    }

}

```

recursive constructor invocation

ex.4

```

class Chess extends IndoorGame{

    Chess(){

        this(12); //call Chess(int x) constructor which accepts int as an arguments within same class

        System.out.println("Chess")

    }

    Chess(int x){

        this(); //call Chess() constructor which accepts zero arguments within same class

    }

}

```

```

public class InheritanceDemo{

    public static void main(String[] args){

        Chess c = new Chess(); // output :

        compile-time error → This creates an infinite loop of constructor calls.

    }

}

```

ex.5

// Parent class

class Game{

String type;

Game(){

System.out.println("Game"); // This prints "Game"

}

}

// Child class of Game

class IndoorGame **extends** Game{

IndoorGame(){

// this constructor never been called by the sub class constructor

System.out.println("IndoorGame");

}

IndoorGame(int x){

super(); //--> calls Game() constructor

}

}

// Child class of IndoorGame

class Chess **extends** IndoorGame{

Chess(){

this(12); //call Chess(int x) constructor within class

System.out.println("Chess"); // Prints "Chess"

}

Chess(int x){

super(5); //call IndoorGame(int x) constructor of super class

}

}

public class InheritanceDemo{

public static void main(String[] args){


```
Chess c = new Chess(); // output :
                        Game
                        Chess
}
}
```

ex.6

```
class Chess extends IndoorGame{
    Chess( String s){
        this(12); //call Chess(int x) constructor within class
        System.out.println("Chess");
    }
    Chess(int x){
        super(5); //call IndoorGame(int x) constructor of super class
    }
}
```

```
public class InheritanceDemo{
    public static void main(String[] args){
        Chess c = new Chess(); // output :
        compile-time error → because in Chess class there is no parameter less constructor is
        created
    }
}
```

ex.7

```
// Child class of Game
class IndoorGame extends Game{
    IndoorGame( String s){
        System.out.println("IndoorGame");
    }
}
```

```
// Child class of IndoorGame
```

```
class Chess extends IndoorGame{
```

```
    Chess(){
```

```
// super() → doesn't call IndoorGame class constructor - because there is no zero argument constructor present in that class .
```

```
// want to call it explicitly with super(args) -->because IndoorGame class only has parameterized constructor
```

```
        System.out.println("Chess");
```

```
    }
```

```
}
```

```
public class InheritanceDemo{
```

```
    public static void main(String[] args){
```

```
        Chess c = new Chess(); // output :
```

```
            compile-time error → IndoorGame doesn't have zero-argument constructor, only IndoorGame(String s).
```

```
    }
```

```
}
```

note:

- If a class has **only a parameterized constructor**, Java compiler **won't create a zero argument constructor** in a class.
 - **super()** works **only if the parent has a Zero argument constructor**.
 - To call a **parameterized constructor** in parent class , you must use **super(arguments)** explicitly.
-

Quiz :

what are all implicitly provided in a class ?

1. class A{

}

ans : class A extends Object{

A(){

super();

}

}

2. class B{

void B(){

}

}

ans : class B extends Object{

B(){

super();

}

void B(){

}

}

3.class C extends D {

C (int x){

super();

}

}

ans : nothing will be provided implicitly
